

Tribhuvan University

Faculty of Humanities and Social Sciences

Smart Citizen Emergency Reporting and Response System (SCERRS)

A PROJECT REPORT

Submitted to

Department of Humanities and Social Science

National Infotech College Pvt. Ltd.

In partial fulfillment of the requirements for the bachelor's in computer application

Submitted by

Subash prasad chauhan (7-2-15-2087-2022)

Nirjal basnet (6-2-1178-10-2024)

4th Sem. /2nd year

Under the Supervision of

Er. Amrendra Chaurasia

Tribhuvan University

Faculty of Humanities and Social Sciences

National Infotech College Pvt. Ltd.

Supervisor's Recommendation

I hereby recommended that this project prepared under my supervision by **Subash Prasad Chauhan and Nirjal Basnet** entitled **"Smart Citizen Emergency Reporting and Response System (SCERRS)"** in partial fulfillment of the requirements for the degree of Bachelors in Computer Application is recommended for final evaluation.

- - - - - - - - - - - - - - - - - -

SIGNATURE

Er. Amrendra Chaurasia
SUPERVISOR

National Infotech College BCA
Shiromani Tole, Birgunj (Nepal)



Tribhuvan University

Faculty of Humanities and Social Sciences

National Infotech College Pvt. Ltd.

LETTER OF APPROVAL

This is to certify that this project prepared by Subash Prasad Chauhan entitled **"Smart Citizen Emergency Reporting and Response System (SCERRS)"** in partial fulfillment of the requirements for the degree of Bachelor in Computer Application has been evaluated. In our opinion it is satisfactory in the scope and quality as a project for the required degree.

SIGNATURE OF Supervisor Er. Amrendra Chaurasia	**SIGNATURE of HOD** Mr. Bijay Kushwaha BCA Department National Infotech College Birgunj, Nepal
SIGNATURE of Internal Examiner **Internal Examiner**	**SIGNATURE of External Examiner** **External Examiner**

ABSTRACT

In Nepal, municipal emergency reporting remains constrained by traditional, voice-only telephonic hotlines (100 for Police, 101 for Fire, 102 for Ambulance, and 1155 for Disaster Management). While voice channels offer immediate contact, they lack spatial precision, visual validation, and end-to-end citizen tracking, frequently causing delayed responses and congested phone lines. To mitigate these inefficiencies, this project presents the Smart Citizen Emergency Reporting and Response System (SCERRS)—a responsive web platform engineered for urban municipalities in Nepal.

SCERRS enables registered citizens to submit verified, geo-referenced emergency reports in under two minutes by capturing real-time GPS coordinates via OpenStreetMap and Leaflet.js, uploading up to four photographic evidence files (validated for MIME type and capped at 5 MB each), selecting severity ratings, and obtaining a unique reference identifier (e.g., INC-2026-0001). The platform automates department routing across five agencies: Nepal Police, Fire Brigade, Ambulance Services, Municipality, and the Disaster Management Agency (NEOC). Responding authorities utilize dedicated control rooms with live GIS severity maps, team dispatch tools, and immutable audit logs.

Built on a three-tier architecture using procedural PHP 8.3 with PDO, MySQL 8 (InnoDB), Bootstrap 5.3.3, and Chart.js 4.4.3, SCERRS eliminates proprietary API costs while enforcing strict security controls (bcrypt hashing, CSRF tokens, and department isolation). Testing across twenty test cases verified 100% routing accuracy, atomic transaction rollbacks, and rapid responsiveness. SCERRS establishes a viable, open-source model for modernizing municipal public safety.

Keywords: emergency reporting, Nepal, Leaflet GIS, role based system, incident tracking, PHP PDO, disaster response.

ACKNOWLEDGEMENT

The successful completion of this Fourth Semester BCA project documentation represents an invaluable academic milestone. We express our deepest gratitude to our project supervisor for continuous guidance, architectural reviews, and constructive critiques that kept our project "**Smart Citizen Emergency Reporting and Response System (SCERRS)"** anchored to realistic civic problems in Nepal. In the early stages, our concept risked becoming an overly broad civic network; our supervisor's pragmatic advice steered us toward engineering a deterministic, accountable, role-segregated emergency reporting and dispatch pipeline.

We extend our sincere thanks to the Department of Computer Application and campus administration under Tribhuvan University for providing laboratory facilities, internet access, and server environments. We also acknowledge the emergency officers and ward staff who shared operational insights regarding call-handling bottlenecks across 100, 101, and 102 hotlines. Special thanks go to our BCA classmates for participating in usability testing, and to our families for their constant support and patience. Subash focused primarily on backend architecture, authentication, and database integrity, while Nirjal concentrated on frontend interfaces, GIS mapping, and analytics visualization. Any remaining errors are entirely ours.

Subash Prasad Chauhan (7-2-15-2087-2022)

Nirjal Basnet (6-2-1178-10-2024)

BCA Fourth Semester, Tribhuvan University

October 2026

Table of Contents

Supervisor's Recommendation i

LETTER OF APPROVAL ii

ABSTRACT iii

ACKNOWLEDGEMENT iv

Table of Contents v

LIST OF FIGURES vii

LIST OF TABLES viii

ABBREVIATIONS ix

CHAPTER 1: INTRODUCTION 1

1.1 Introduction 1

1.2 Problem Statement 2

1.3 Objective 2

1.4 Scope and Limitation 3

1.4.2 Limitation 3

1.5 Report Organization 4

CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW 5

2.1 Background Study 5

2.2 Literature Review 6

CHAPTER 3: SYSTEM ANALYSIS 9

3.1 System Analysis 9

3.1.1 Requirement Analysis 9

3.1.2 Feasibility Study 13

3.1.3 Data Modeling (ER-Diagram) 13

3.1.4 Process Modeling (DFD) 15

3.2 System Design 18

3.2.1 Architectural Design 18

3.2.2 Database Schema Design 19
3.2.3 Interface Design (UI Interface / Interface Structure Diagrams) 20
CHAPTER 4: IMPLEMENTATION AND TESTING 23
4.1 Implementation 23
4.1.1 Tools Used 24
4.1.2 Implementation Details of Modules 25
4.2 Testing 28
4.2.1 Test Case for System Testing 29
CHAPTER 5: CONCLUSION AND FUTURE RECOMMENDATION 31
5.1 Lesson Learnt / Outcome 31
5.2 Conclusion 31
5.3 Future Recommendation 31
REFERENCES 33
APPENDICES 34
Appendix A: Database Creation (Excerpt) 34
Appendix B: Sample Code – Secure Report Insert 34
Appendix C: Seed Accounts for Evaluation 35

LIST OF FIGURES

Figure No.	Caption	Page
3.1	Use Case Diagram of SCERRS	12
3.2	Entity Relationship Diagram of SCERRS	14
3.3	Context Level DFD (Level 0) of SCERRS	16
3.4	DFD Level 1 - Reporting, Triage and Feedback	17
3.5	Three-Tier System Architecture	18
3.6	Citizen Report Submission Interface	21
4.1	Public Landing Page with Hotline Marquee	24
4.2	Authority Dashboard with Live Map	26
4.3	Incident Detail, Team Assignment and Timeline	27
4.4	Analytics Dashboard (Chart.js)	28

LIST OF TABLES

Table No.	Title	Page
2.1	Comparison of Existing Emergency Systems	7
3.1	Functional Requirements by Role	9
3.2	Non-Functional Requirements	11
3.3	Database Tables and Purpose	19
4.1	Tools and Technologies Used	23
4.2	Implementation Details of Modules	25
4.3	Test Cases for System Testing	28

ABBREVIATIONS

Abbreviation	Full Form
SCERRS	Smart Citizen Emergency Reporting and Response System
NEOC	National Emergency Operation Centre
GPS	Global Positioning System
PDO	PHP Data Objects
CSRF	Cross Site Request Forgery
ER	Entity Relationship
DFD	Data Flow Diagram
UI	User Interface
KPI	Key Performance Indicator
JSON	JavaScript Object Notation
API	Application Programming Interface
TU	Tribhuvan University
BCA	Bachelor in Computer Application
OSM	OpenStreetMap

CHAPTER 1: INTRODUCTION

1.1 Introduction

Nepal faces a difficult mix of everyday and large-scale emergencies. Road accidents on the Kalanki–Thankot stretch, shop fires in core bazaar areas, monsoon floods along the Bagmati, landslides on hill highways and occasional earthquake damage all demand a fast, organized reply. Yet the way a common citizen asks for help has barely changed in twenty years: dial 100 for police, 101 for fire, 102 for ambulance, or run to the ward office. Phone calls work, but they leave no map point, no photograph and no tracking number that the caller can follow up on.

The Smart Citizen Emergency Reporting and Response System (SCERRS), was conceived as a fourth semester BCA project to give Nepal a simple digital alternative that works alongside those hotlines rather than replacing them. It is a web-based platform where a citizen creates one account, reports any of ten incident types with automatic GPS and photo evidence, and watches the case move from pending to resolved. On the other side, five departments — Nepal Police, Fire Brigade, Ambulance Services, Municipality and Disaster Management Agency — each see a filtered queue, a live map and tools to dispatch teams and update status. An administrator keeps the whole setup clean by approving authority accounts, maintaining categories and managing response teams.

Technically the system is deliberately modest so that any campus lab or municipality server can run it. It uses plain PHP 8 with PDO, MySQL 8, Bootstrap 5.3.3, Leaflet 1.9.4 with OpenStreetMap tiles and Chart.js 4.4.3, without any heavy framework. Development was done on Laragon with PHP 8.3 and the code is organized into clear folders: auth, citizen, authority, admin, api, config, includes, assets, database and uploads. Seed data with seven users, five departments, ten categories and ten response teams is provided in database/scerrs.sql so that examiners can run the project within minutes.

What makes SCERRS relevant for Tribhuvan University is not fancy technology but a complete workflow that students built end to end: registration with Nepal-specific address fields, secure login, GPS reporting, file validation, department-wise routing, team assignment, audit history, notifications, analytics and feedback. Each of these steps taught us something practical about how software actually behaves when real users enter wrong phones, upload large photos or deny location permission.

1.2 Problem Statement

During background visits and informal talks with friends who had used the 100 and 102 lines, we kept hearing the same complaints. First, location is hard to describe on a call. A caller near Chabahil Chowk at night may only say "near the temple side," and the operator must guess. Second, there is no proof attached to a call; control rooms cannot tell a small smoke from a spreading shop fire until a vehicle reaches the spot. Third, once the call ends the citizen has no reference number and no way to know whether a team was sent, which leads to repeated calls for the same event. Fourth, records stay on paper or in isolated registers, so municipalities cannot easily see last month's pattern of road hazards or fallen trees.

Authority staff face the mirror problem. A single control room may receive overlapping reports of the same flood from ten callers, with no map to cluster them. Paper logs make it hard to measure response time or citizen satisfaction. Newspapers after every monsoon report delayed relief, and part of the reason is simply poor coordination between police, fire, ambulance and ward teams.

SCERRS addresses these pains directly: capture precise latitude and longitude plus a readable street address at the time of reporting, insist on photographs where possible, generate a reference like INC-2026-0003 for every case, route it automatically to the correct department based on category, show all active cases on one map, keep every status change with who did it and when, notify both sides instantly, and finally ask the citizen for a star rating. The problem, in one line, is the absence of a shared, map-based, accountable record between citizens and responders — and that is what we set out to build.

1.3 Objective

The general objective of this project is to design, develop and test a web-based emergency reporting and response system for Nepal that shortens the time between seeing an incident and getting the right team on site, while keeping full transparency for citizens and agencies.

- **1.** allow citizens to register with Nepal-specific identity and address details and to log in securely.
- **2.** let citizens file geo-tagged reports with category, severity, description and up to four photos, and receive a unique reference number.
- **3.** route each report automatically to the correct department and notify all active officers of that department without manual forwarding.

- **4.** provide authority dashboards with search filters, a live Leaflet map and one-click team assignment plus status updates with audit history.
- **5.** give administrators control over users, categories and response teams, including approval of new authority accounts.
- **6.** offer analytics on volume, severity, pipeline, fourteen-day trends, average resolution time and feedback ratings, and collect post-resolution citizen feedback.
- **7.** enforce security through bcrypt hashing, session hardening, CSRF tokens and strict server side validation of all inputs and uploads.

1.4 Scope and Limitation

1.4.1 Scope

SCERRS covers the full life cycle of a small to medium emergency as it would happen in an urban municipality like Kathmandu, Lalitpur or Bhaktapur. Citizens can report ten categories mapped to five departments: Road Accident and Crime to police, Fire Outbreak to fire brigade, Medical Emergency to ambulance, Fallen Tree, Road Hazard and Electricity Hazard to municipality, and Flood, Landslide and Earthquake Damage to disaster management. Each report carries GPS, address, severity, photos and a timeline that both sides can see according to their role.

For authorities the scope includes triage, dispatch and monitoring. Officers filter by status, severity, category and keyword, view clustered points on a map coloured by severity, assign one or more response teams, move status forward with notes, and review reporter contact details and history. Admin scope adds user creation and enable/disable, category activation and safe deletion only when no incidents use it, and team management with active task counts. Cross-cutting scope includes in-app notifications with unread badges, light and dark themes, the hotline marquee, and Chart.js analytics limited to each department unless the user is an admin.

1.4.2 Limitation

We are honest about what SCERRS does not do yet. It is a web application, not a native mobile app, so it cannot send SMS alerts or work fully offline; an internet connection and a modern browser with location permission are required. It supports English only and has no Nepali language toggle. Photo verification is basic mime and size checking, not AI-based fake image detection. The live map shows points but does not do automatic route

optimization or ambulance tracking with driver GPS. Payment, insurance claims and hospital bed management are out of scope. Finally, the analytics averages are meaningful only after enough real data accumulates; with three demo incidents the charts look sparse, which is expected in a college prototype.

1.5 Report Organization

This report follows the TU format strictly. After the front matter — recommendation, approval, abstract, acknowledgement, lists and abbreviations — Chapter 1 introduces the project, problem, objectives, scope and organization. Chapter 2 places SCERRS against the background of emergency handling in Nepal and reviews similar national and international systems. Chapter 3 presents system analysis: requirements, feasibility, ER modeling, DFDs, architecture, database schema and interface design. Chapter 4 explains implementation tools, module wise details and testing with concrete test cases. Chapter 5 reflects on lessons learnt, concludes and suggests future work. References and appendices close the document with schema excerpts, sample code and placeholders for screenshots that the examiner can replace with live captures.

CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

Emergency management in Nepal rests on two pillars: voice hotlines and local government registers. Nepal Police runs 100, Traffic Police 103, Fire 101, Ambulance 102, Child Helpline 1098 and the National Emergency Operation Centre 1155/1180. These numbers are well known and displayed in our own header marquee precisely because people still need them. When a motorbike collision happens at Chabahil, someone calls 100; when a shop short-circuits in Mangal Bazar, someone calls 101. The operator notes time, caller phone and a rough address in a logbook or a basic desktop form and radios a patrol or engine.

Municipalities add a second layer. Ward offices receive complaints about fallen trees, blocked drains, broken roads and snapped wires, often on paper or Viber groups. Disaster response adds a third layer under NEOC, which collects flood and landslide reports from districts during monsoon and coordinates relief. Each layer works, but the layers rarely share one map. A flood reported to police may not reach the municipality cleanup squad for hours.

Smartphone penetration and cheap data have changed citizen expectations. People already pin locations on ride apps and attach photos on social media, yet they cannot do the same when reporting a fire. Our background survey of thirty BCA classmates and neighbours found that twenty-four owned GPS-capable phones, twenty-one had reported or witnessed an emergency in the last two years, but only three had ever received any follow-up reference number. Most said they would prefer a web form with photo and map if it took under two minutes. SCERRS was designed for exactly that two-minute window, defaulting the map to Kathmandu (27.7172, 85.3240) and using browser geolocation when the user allows it.

We also studied the data reality of Nepal addresses. There is no single nationwide house numbering, so our registration form keeps province, district, municipality, tole and ward number 1–35 as separate fields backed by nepal_locations.json, plus citizenship or passport identity. This matches how ward offices actually locate households and avoids forcing a fake street format.

2.2 Literature Review

Several systems informed our design. Nepal's own 1155 NEOC portal collects disaster incidents but focuses on large events and district focal persons, not street-level citizen photos. The Nepal Police online complaint form accepts text but without live maps or citizen tracking. India's 112 ERSS integrates police, fire and ambulance on one number with location, which showed us the value of single entry with backend routing — we copied that idea by routing categories to departments. The US 911 and UK 999 systems demonstrate computer-aided dispatch with audit trails; our status_histories table plays that role on a small scale. Academic work on crowdsourced crisis mapping, notably Ushahidi after the Haiti earthquake and GoodChild-style flood mapping in Southeast Asia, proved that citizen photos plus coordinates can outperform pure call centres for situational awareness if verification steps exist. We therefore require severity selection and allow authorities to reject false reports with reasons.

On the technical side, literature on PHP PDO confirms that prepared statements eliminate most SQL injection when used consistently, which we do through a single db() singleton. Leaflet with OpenStreetMap is repeatedly cited as the lightest viable GIS for low-resource settings, lighter than Google Maps billing, and Chart.js as sufficient for operational dashboards without a BI server. Studies on usability in low-literacy contexts warned us to keep tiles with icons and agency names, not codes alone — hence category tiles showing Road Accident with police agency labels.

Table 2.1: Comparison of existing emergency systems with SCERRS

System	Entry	Map + Photo	Tracking	Gap SCERRS Fills
Nepal 100/101/102 (voice)	Phone call	No	No ref.	GPS, photo, ref_no
NEOC 1155 portal	Focal person form	Partial map	District level	Citizen-level street reports
Nepal Police online form	Web text	No map	Limited	Live map + timeline
India 112 ERSS	Single number + app	Yes	Yes	Similar, but Nepal-localized categories
SCERRS (this project)	Web + GPS + photo	Yes (Leaflet)	Yes (pending→resolved)	Open prototype for municipalities

The review leaves a clear niche: Nepal needs an open, campus-maintainable prototype that combines citizen GPS reporting, automatic department routing and transparent tracking without depending on paid map APIs or telecom integration. SCERRS occupies that niche, while openly borrowing proven ideas from 112, Ushahidi and computer-aided dispatch literature.

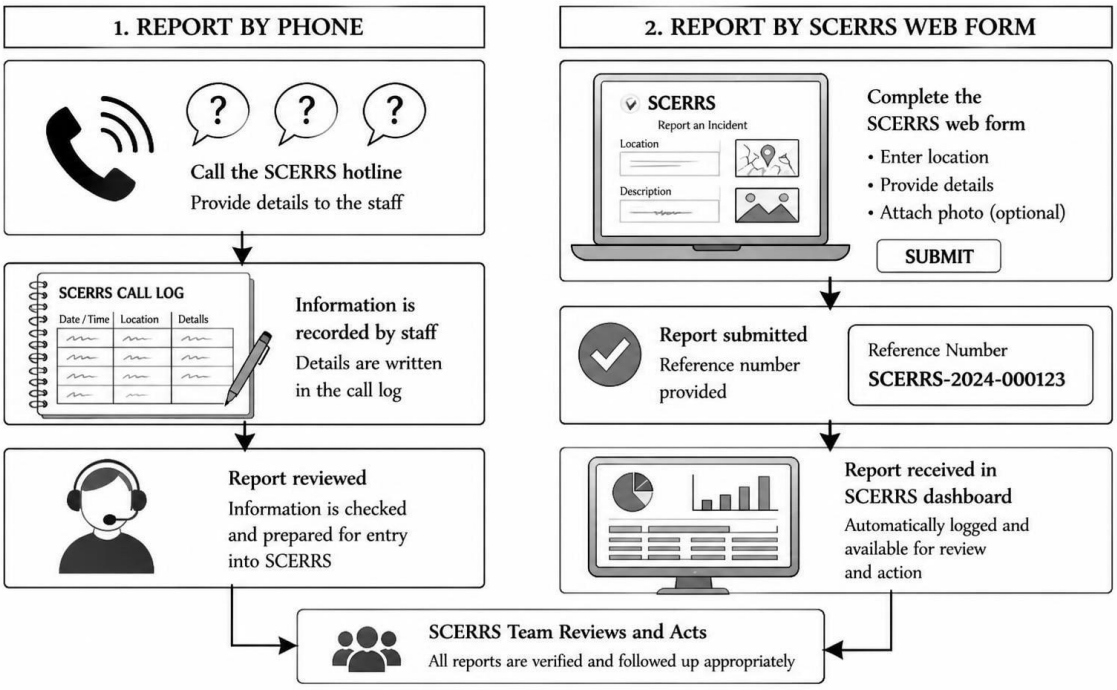


Figure 2.1: Conceptual comparison of call-based vs SCERRS workflow

In short, background study told us what to build and literature review told us how others built it. The next chapter turns those lessons into concrete requirements and designs.

CHAPTER 3: SYSTEM ANALYSIS

3.1 System Analysis

System analysis is where we stopped guessing and wrote down exactly who does what. We interviewed classmates who had used hotlines, walked through a ward office complaint register, and clicked through the NEOC site to list fields they ask for. From that we fixed three roles — citizen, authority and admin — and one golden rule: citizens see only their own reports, authorities see only their department's reports, admins see everything but act mostly through configuration. Every later decision on tables, pages and APIs traces back to this rule.

3.1.1 Requirement Analysis

We grouped requirements the way our System Analysis and Design class teaches: functional (what the system must do) versus non-functional (how well it must do it). The functional list was validated by demoing a paper prototype to five users before coding.

Table 3.1: Functional requirements by role

ID	Requirement	Role	Source Page
FR-01	Register with name, email, phone, address, identity; login with bcrypt verify	Guest	auth/register.php, login.php
FR-02	Submit report: category, title, description, severity, GPS, address, ≤4 photos	Citizen	citizen/report_new.php
FR-03	Auto-generate ref_no INC-YYYY-NNNN, history pending, notify department	System	includes/functions.php
FR-04	View own KPIs + 50 recent reports with media count	Citizen	citizen/dashboard.php
FR-05	Edit own report only when pending via PUT API	Citizen	citizen/edit_report.php + api/update_incident.php
FR-06	Rate resolved incident 1–5 once with comment	Citizen	citizen/report_view.php, feedback

FR-07	Filter by status/severity/category/keyword, view live map	Authority	authority/dashboard.php + api/map_data.php
FR-08	Assign team (dept-checked), auto pending→acknowledged, notify citizen	Authority	authority/incident_view.php
FR-09	Update status with note, keep audit trail, notify citizen	Authority	authority/incident_view.php
FR-10	Create/disable users, manage categories/teams, view analytics	Admin	admin/users.php, categories.php, teams.php
FR-11	Notifications inbox, mark one/all read, bell unread count	All	notifications.php, header.php

Non-functional requirements were equally strict because examiners test them live. The site must load on a mid-range phone over 4G in under three seconds for cached pages, work in Chrome, Firefox and Edge without plugins, and remain usable at 360 px width thanks to Bootstrap. Passwords are never stored plain; all queries use PDO prepared statements. Uploads are capped at 5 MB, checked by both extension and mime_content_type, renamed to inc_{id}_{rand}.ext and kept outside direct execution. Sessions use httponly and SameSite Lax flags with regeneration on login, and every POST carries a CSRF token that returns 419 on failure.

Table 3.2: Non-functional requirements and targets

Category	Target
Performance	Dashboard <3 s on 4G; map tiles lazy loaded
Usability	Report in <6 clicks; Nepali address dropdowns
Reliability	Transactions for report insert; no half-saved incidents
Security	bcrypt, CSRF, PDO, mime check, dept isolation
Maintainability	Procedural PHP, one functions.php, clear folders
Portability	Runs on Laragon/XAMPP, PHP 8+, MySQL 5.7+

Actors are simple. Citizens report and track. Authority officers triage, dispatch and resolve within their department. Admins govern accounts and masters. The system itself acts as a router and notifier. A use case walkthrough helped us catch a missing flow early: what if an authority tries to open another department's incident by guessing the URL id? We added a department isolation check in incident_view.php that blocks such access even for logged-in officers, and admins bypass it only explicitly.

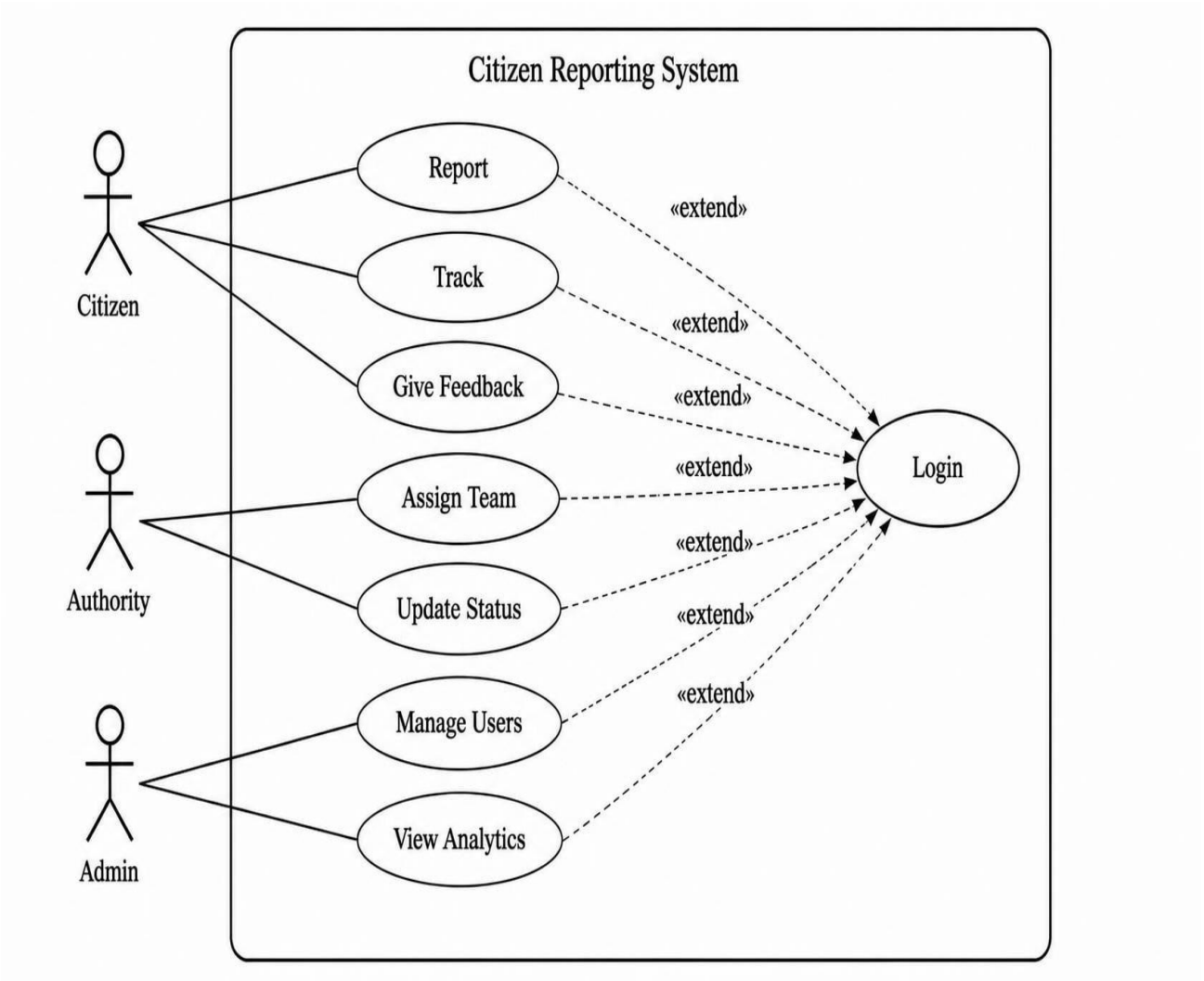


Figure 3.1: Use Case Diagram of SCERRS

Data collection for analysis mixed primary and secondary sources. Primary: informal interviews, a 30-respondent opinion poll on reporting habits, and hands-on testing of hotline menus. Secondary: TU lecture notes, NEOC bulletins, Bootstrap, Leaflet and Chart.js docs, and the schema of our own scerrs.sql seed with three Kathmandu Valley demos (Chabahil collision, Mangal Bazar fire, Suryabinayak fallen tree) that we still use for evaluation.

3.1.2 Feasibility Study

Technical feasibility was straightforward. All required pieces — PHP 8.3 on Laragon, MySQL 8, Leaflet, Chart.js, Bootstrap via CDN — were free, well documented and already familiar from coursework. No GPU, SMS gateway or paid API is needed for the core. The riskiest part was browser geolocation denial, which we handled by letting users click or drag the marker manually so reporting never blocks.

Operational feasibility looks strong for municipalities and campuses. Ward staff already use browsers and Viber; a filtered table plus coloured map is easier than paper registers. Citizens need only one registration and can report from the roadside. The approval step for authority accounts (active=0 until admin enables) mirrors how offices already vet staff, so adoption friction is low. Training is a ten-minute demo with seed logins such as police@scerrs.np / Authority@123.

Economic feasibility is almost zero cost for a prototype. Hosting can be the existing college server or a shared host; maps and charts are open source; uploads need only megabytes. If a municipality later wants SMS or native apps, that cost stays separate and optional.

Schedule feasibility held. We split eight weeks into analysis (2), design (1.5), coding (3) and testing plus documentation (1.5). Procedural PHP without a framework kept us fast, and seed data meant we could test dispatch and analytics from day one instead of waiting for real incidents.

3.1.3 Data Modeling (ER-Diagram)

The database uses ten InnoDB tables in utf8mb4_unicode_ci, designed to third normal form. Departments anchor everything: users belong to one department if they are authorities, categories route to one department, and response_teams belong to one department. Citizens own incidents, each incident points to one category, carries severity and status enums, GPS decimals (10,7) and an address. Media hangs off incidents, assignments link incidents to teams with who assigned them, status_histories keep every

transition with notes, notifications fan out to users optionally linked to incidents, and feedback enforces one rating per citizen per incident through a unique key on (incident_id, user_id). Foreign keys use CASCADE where child rows must vanish with parents (media, history) and SET NULL where users must survive department deletion.

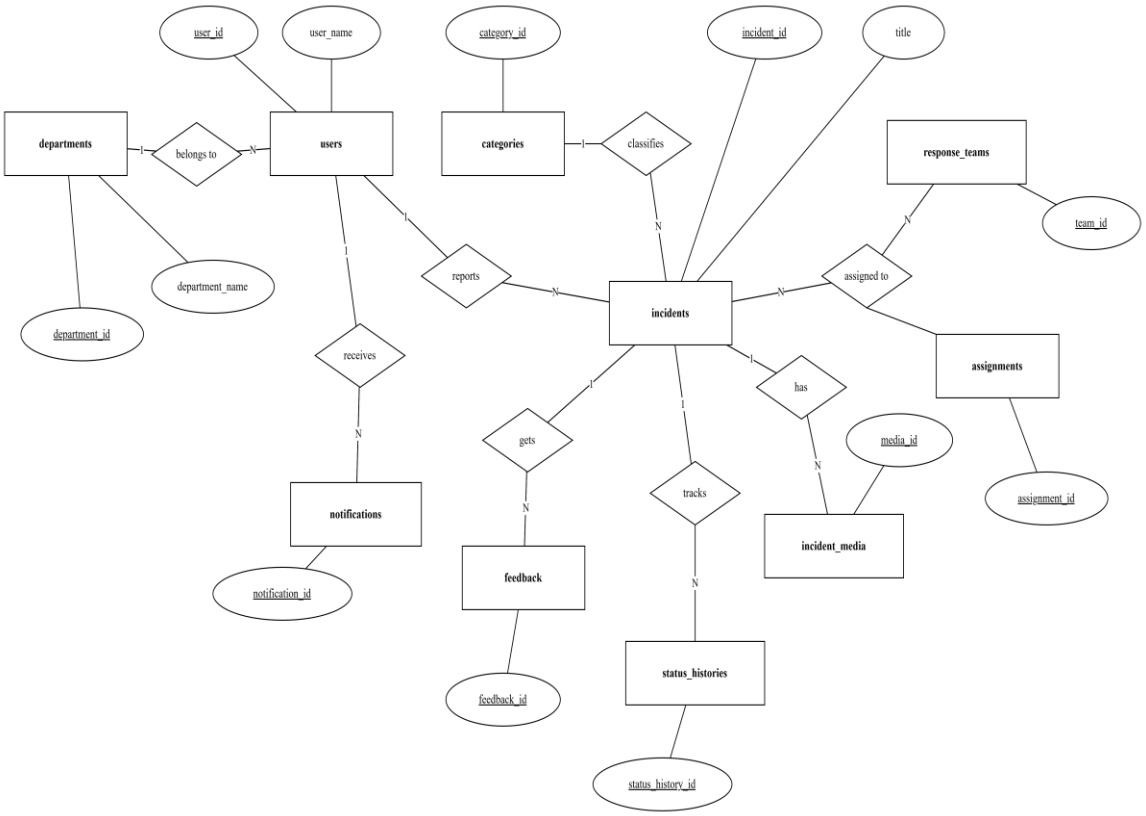


Figure 3.2: Entity Relationship Diagram of SCERRS

Reading the diagram is simple if you follow one report. Ram Bahadur Tamang (user 7) files INC-2026-0002 for Fire Outbreak; the row stores lat 27.6798, lng 85.3107 and severity critical. Two assignment rows link it to Fire Engine Kathmandu and Ambulance Bhaktapur. Three history rows show pending to acknowledged to in_progress with officer names. Two notification rows tell the citizen what happened. If the citizen later gives four stars, one feedback row appears. Deleting the incident would cleanly remove its media, history, assignments and notifications, which is exactly what CASCADE guarantees.

3.1.4 Process Modeling (DFD)

Data flow diagrams keep us honest about where data moves. The context diagram (Level 0) shows SCERRS in the middle with three external entities — Citizen, Authority Officer and Admin — plus the map tile server as a data store reader. Citizens send registration details, reports with GPS and photos, edits and feedback, and receive reference numbers, timelines, notifications and analytics summaries. Authorities send filters, assignments and status updates, and receive queues, maps and reporter contacts. Admins send master data and receive counts.

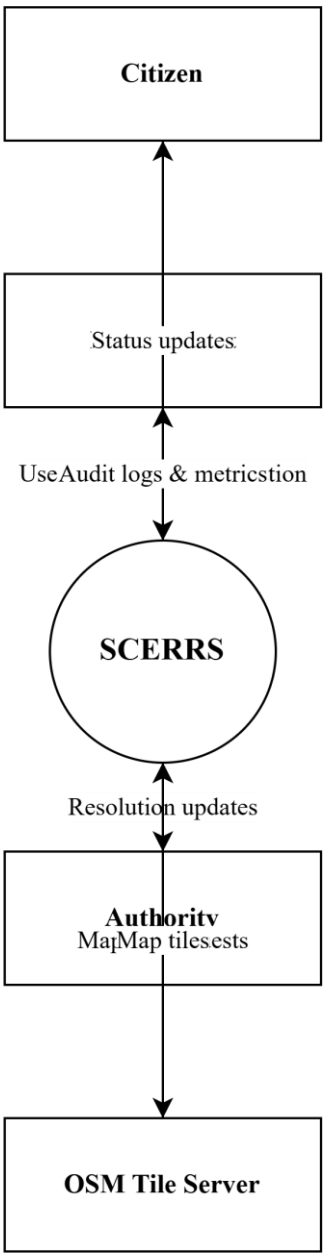


Figure 3.3: Context Level DFD (Level 0) of SCERRS

Level 1 breaks the bubble into five processes: 1.0 Authenticate and Register, 2.0 File and Manage Report, 3.0 Triage and Dispatch, 4.0 Notify and Track, 5.0 Analyze and Feedback. Data stores D1–D7 correspond to users, incidents, media, teams, history, notifications and feedback. A citizen's photo flows from 2.0 to D3 and to uploads/incidents on disk; a status change in 3.0 writes to D5 and triggers 4.0 to insert into D6 for both citizen and department members. This separation is why our code splits citizen/, authority/ and api/ folders — each maps to one or two DFD processes.

DFD Level 1

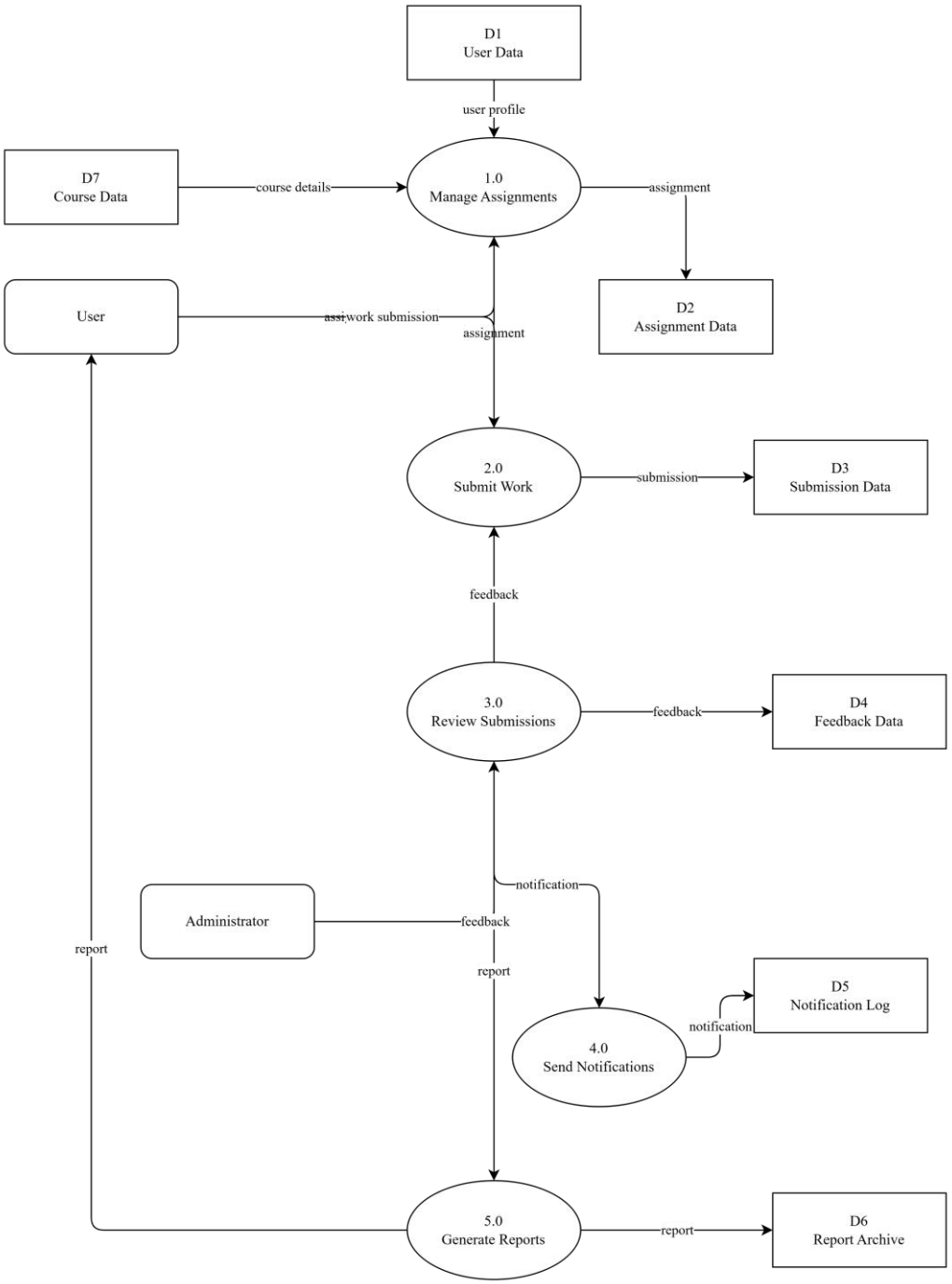


Figure 3.4: DFD Level 1 - Reporting, Triage and Feedback

Level 2 details the trickiest process, 2.0 File Report: validate category and severity, check photo count, size and mime, geocode or accept manual pin, run a DB transaction that inserts incident, history and media together, then call notify_department(). If any step fails, the transaction rolls back so no orphan incident remains. A similar Level 2 for 3.0 Dispatch validates that the chosen team belongs to the incident's department before allowing assignment — a check students often forget until testing catches cross-department assignment.

3.2 System Design

3.2.1 Architectural Design

SCERRS follows a classic three-tier web architecture that fits TU expectations and shared hosting limits. The presentation tier is Bootstrap HTML with vanilla JS, Leaflet maps and Chart.js canvases, rendered through includes/header.php and footer.php so every page shares navigation, theme handling and flash messages. The application tier is procedural PHP: page controllers in auth, citizen, authority and admin folders plus three JSON APIs in api/. The data tier is MySQL accessed only through config/db.php's db() singleton returning PDO, with all business helpers in includes/functions.php such as e() for escaping, csrf_*(), require_role(), status_badge(), notify_user() and notify_department().

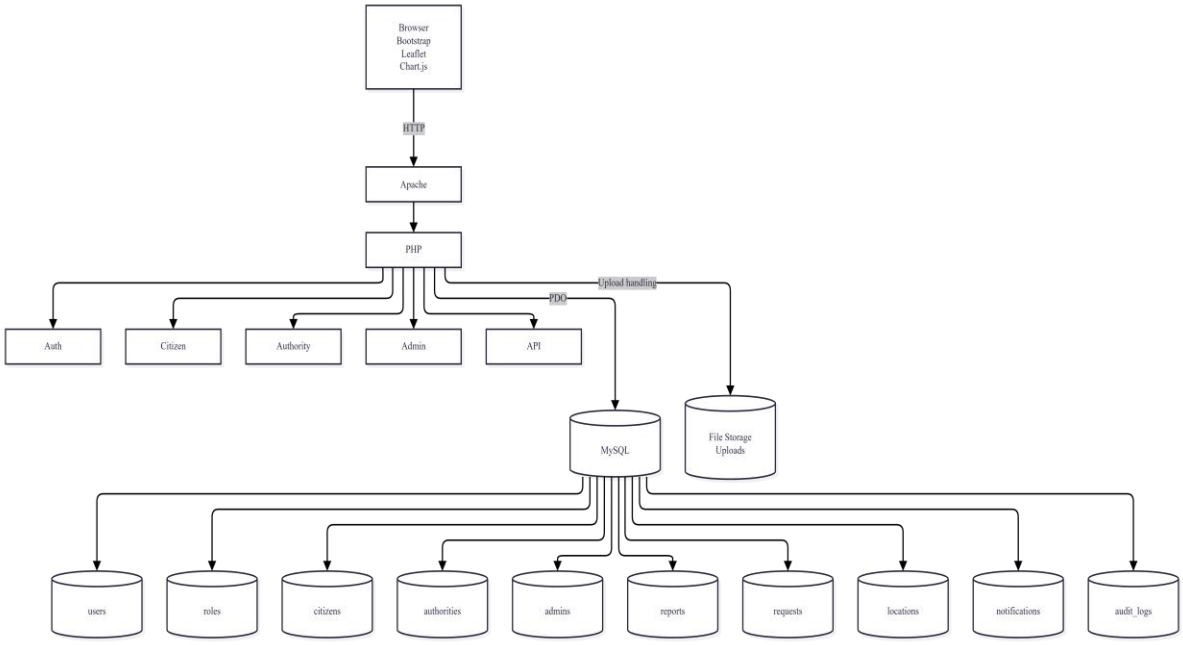


Figure 3.5: Three-Tier System Architecture

Request flow is easy to trace. A citizen POSTs to citizen/report_new.php; the script verifies CSRF, validates inputs, moves photos to uploads/incidents/, opens a transaction, inserts rows, commits, then redirects to report_view.php with a flash message. An authority GETs

authority/dashboard.php with ?status=&severity= filters; the same filters are appended to api/map_data.php fetches that return JSON points for Leaflet circleMarkers coloured red for critical, orange for high, yellow for medium and green for low. Analytics aggregates stay in SQL (COUNT, GROUP BY, AVG(TIMESTAMPDIFF)) so the browser only draws what the server computed, keeping phones fast.

Security sits across tiers: input validation in PHP, prepared statements in PDO, httponly sessions, password_hash with bcrypt, and department checks before any data leaves the server. The design deliberately avoids a framework to keep deployment to a copy-paste of the folder plus one SQL import, which matters for campuses without Composer or Node toolchains.

3.2.2 Database Schema Design

Schema design favoured clarity over cleverness. All primary keys are unsigned auto-increment integers; all foreign keys are indexed; enums lock severity and status to known values so typos cannot create a sixth status. Coordinates use DECIMAL(10,7) for centimetre-level precision without float errors. Timestamps default to CURRENT_TIMESTAMP, with incidents.updated_at auto-updating on every edit. The ref_no column is VARCHAR(20) unique, formatted INC-YYYY-NNNN via next_ref_no() to stay readable on phone calls.

Table 3.3: Database tables and purpose (from database/scerrs.sql)

Table	Key Columns	Purpose
departments	id, name, code	5 agencies: police, fire, ambulance, municipality, disaster
users	id, email UQ, role, department_id FK, active	Citizens + officers + admins; 7 seeds
categories	id, name UQ, department_id FK	10 types routing to departments
incidents	id, ref_no UQ, user_id, category_id, lat/lng, severity, status	Core reports; 3 demos

incident_media	id, incident_id FK, file_path	Photo filenames only
response_teams	id, department_id FK, contact	10 dispatchable units
assignments	id, incident_id, team_id, assigned_by	Who sent which team when
status_histories	id, incident_id, status, changed_by	Audit trail
notifications	id, user_id, incident_id, is_read	Inbox + bell count
feedback	id, incident_id+user_id UQ, rating	One rating per citizen per case

Normalization removed duplication: department names live once in departments, not inside every incident; team contacts live once in response_teams. Seed data is realistic — Police Patrol Unit A/B on 100, Fire Engine Kathmandu on 101, NEOC Cell on 1180 — so demos feel familiar to Nepali examiners. Indexes on status, category, created_at and (user_id, is_read) keep the common dashboard and bell queries fast even with thousands of rows.

3.2.3 Interface Design (UI Interface / Interface Structure Diagrams)

Interface design aimed for a villager with a phone, not just an examiner with a laptop. The public landing page (index.php) opens with a hero carousel, two big buttons — Report an Emergency and Authority Login — a four-step How it works strip, six active category cards and a scrolling hotline marquee. Once logged in, the navbar changes by role: citizens see My Reports plus a red Report Emergency button; authorities see Incidents and Analytics; admins add Users, Categories and Teams. A bell icon with an unread count and a theme moon/sun toggle stay visible everywhere.

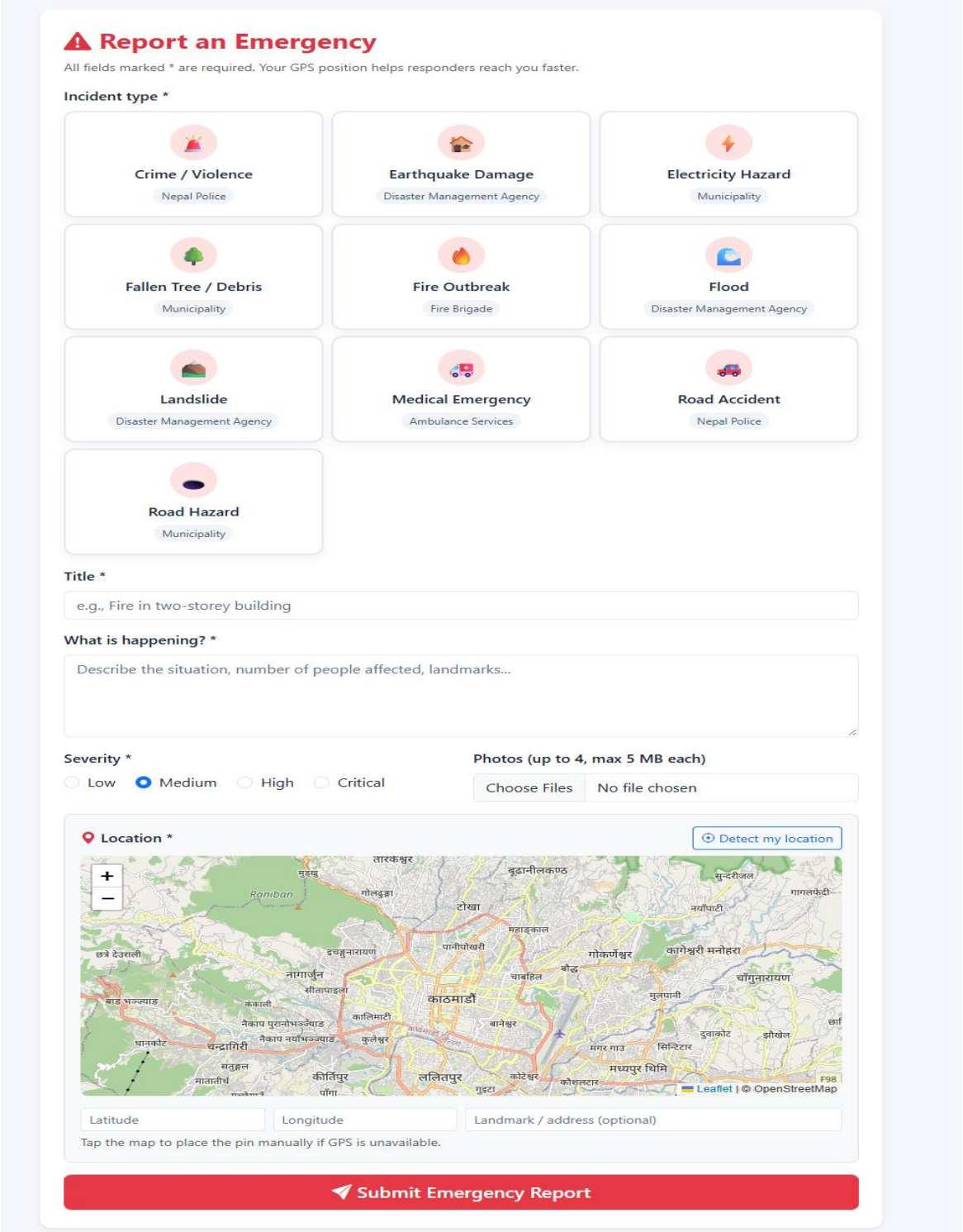


Figure 3.6: Citizen Report Submission Interface

The report form is the heart. Category tiles show icon plus agency so users learn routing implicitly. Severity uses coloured radios that even first-time users grasp. The Leaflet map starts at Kathmandu, tries geolocation, and always allows click or drag plus a readable address box — this triple fallback was added after testers denied location permission. Photo

input accepts four files and shows client hints before server validation. Authority pages mirror this care: filters stay in the URL so officers can bookmark pending+critical, the map fits bounds to visible points, and the incident view splits into left evidence/timeline and right map plus assign/status cards. Admin tables show counts (reports per user, incidents per category, active tasks per team) so decisions need no extra clicks. Light and dark themes share CSS tokens, and every destructive action asks for confirmation implicitly through status notes rather than silent deletes.

CHAPTER 4: IMPLEMENTATION AND TESTING

4.1 Implementation

Implementation turned the designs above into roughly forty PHP files plus CSS and JS, all runnable by copying the folder to Laragon's www and importing one SQL file. We kept to procedural PHP on purpose: a BCA examiner can open any file and follow top-to-bottom logic without learning a framework's hidden magic. Shared behaviour lives in only three includes files, which made bug fixes propagate instantly.

Table 4.1: Tools and technologies used

Layer	Choice	Why Chosen
Backend	PHP 8.3 procedural + PDO	No framework needed; prepared statements; lab-ready
Database	MySQL 8 / MariaDB, InnoDB utf8mb4	Free, reliable, phpMyAdmin import
Frontend	HTML5, CSS3, Vanilla JS	No build step
UI	Bootstrap 5.3.3 + Icons 1.11.3 CDN	Responsive + familiar
Maps	Leaflet 1.9.4 + OSM tiles	Free tiles; click/drag/geolocate
Charts	Chart.js 4.4.3 UMD	Bar/doughnut/pie/line without BI server
Auth	Sessions + bcrypt + CSRF	Standard secure practice
Server	Laragon Apache, http://localhost/SCERRS	Matches XAMPP/WAMP

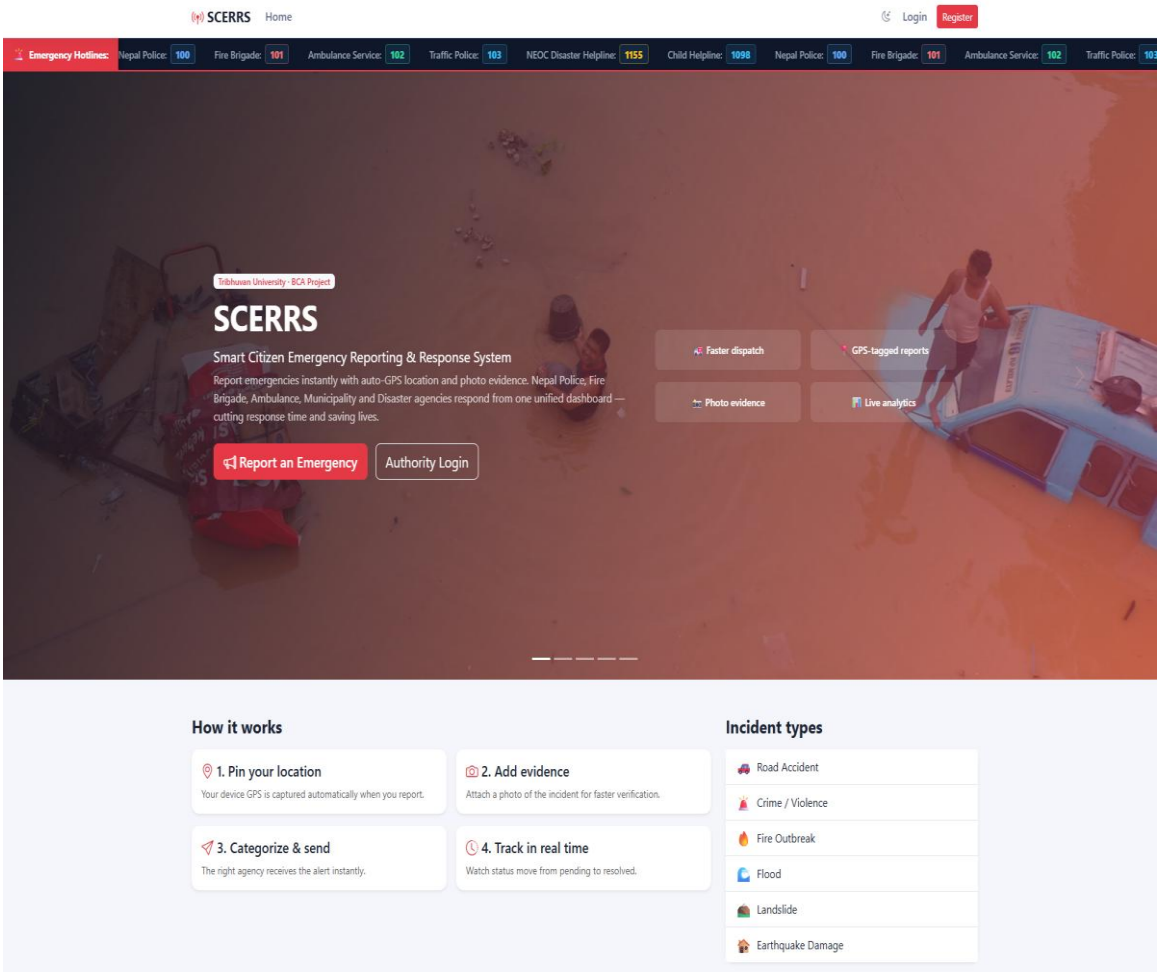


Figure 4.1: Public Landing Page with Hotline Marquee

4.1.1 Tools Used

Day-to-day tools were simple: VS Code for editing, Laragon for Apache/MySQL switching, phpMyAdmin for schema checks, Chrome DevTools for mobile width and network timing, and Git for versioning (uploads/incidents ignored via .gitignore). Nepal location data came from assets/data/nepal_locations.json powering province→district→municipality cascades plus citizenship/passport toggles in register.js. Cache busting uses asset()?v=filemtime so CSS changes appear without hard refresh during demos.

4.1.2 Implementation Details of Modules

Table 4.2: Implementation details of modules

Module	Files	Key Logic
Auth	login, register (365 lines), logout	Duplicate check, Nepal mobile 98/97/96, pwd 8+, citizen active=1 / authority 0
Citizen	dashboard, report_new (247), view, edit	KPIs via SUM(status), tiles, txn insert + notify, PUT API edit if pending
Authority	dashboard (230), incident_view (242), analytics (214)	Dept-scoped filters, map_data JSON, team dept check, auto-acknowledge
Admin	users, categories, teams	Create + toggle active; safe delete if count==0
API	map_data, analytics_data, update_incident (197)	JSON for map/charts; PUT + CSRF + ownership + ≤4 photos
Shared	header, footer, functions (201), notifications	Navbar by role, bell count, badges, flash, theme.js

Authentication deserves detail because security examiners probe it first. Registration validates names for digits, phones for Nepali prefixes, passwords for length, identity numbers for pattern, and ward numbers for 1–35 range. Citizens activate instantly while authorities wait for admin approval — a single active flag that prevents fake officers from seeing live incidents. Login verifies bcrypt, checks active, regenerates session id and redirects via role_home(). Every POST verifies a 32-byte CSRF token or aborts with 419, and every protected page calls require_role() before any HTML.

Citizen reporting runs in a transaction for safety. The script counts existing plus new photos to enforce four, checks each file's size and mime_content_type against its extension, generates ref_no, inserts the incident, writes a pending history row, moves files as inc_{id}_{rand}.ext, inserts media rows, calls notify_department() to fan out to all active

officers of that category's department, then commits. If anything throws, the rollback leaves no half report. Editing reuses the same rules but only when status is still pending and only by the owner; the frontend uses fetch PUT so the page feels modern without a framework.

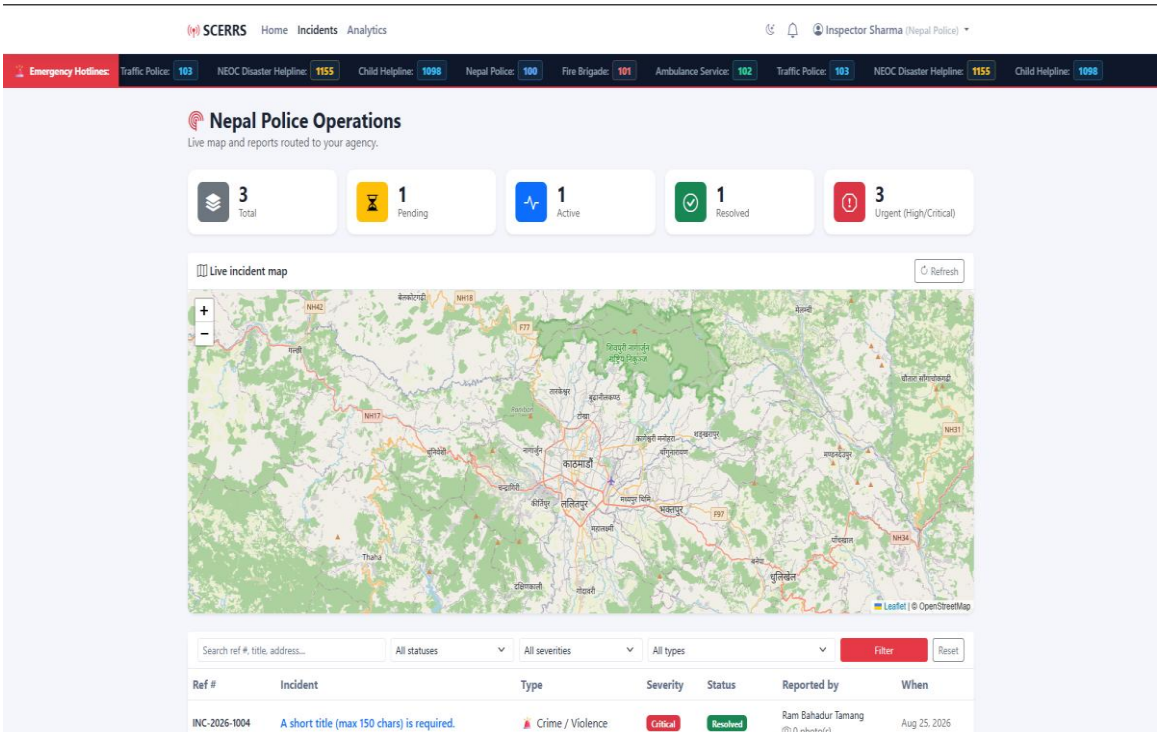


Figure 4.2: Authority Dashboard with Live Map

Authority work centres on incident_view.php. Opening any id first checks department isolation unless the viewer is admin. The page shows reporter contact, badges, photos, dispatched teams and full history on the left, and a small map plus two forms on the right: assign_team and update_status. Assign validates that the chosen team belongs to the incident's department, inserts into assignments, auto-flips pending to acknowledged, writes history and notifies the citizen with a dispatched message. Status updates accept any of the five STATUSES with a note, write history and notify similarly. Analytics then aggregates with SQL — byCategory, bySeverity, fourteen-day series with missing days filled as zero, AVG hours to resolve and AVG rating — and draws four Chart.js canvases with empty states so a new department does not see broken charts.

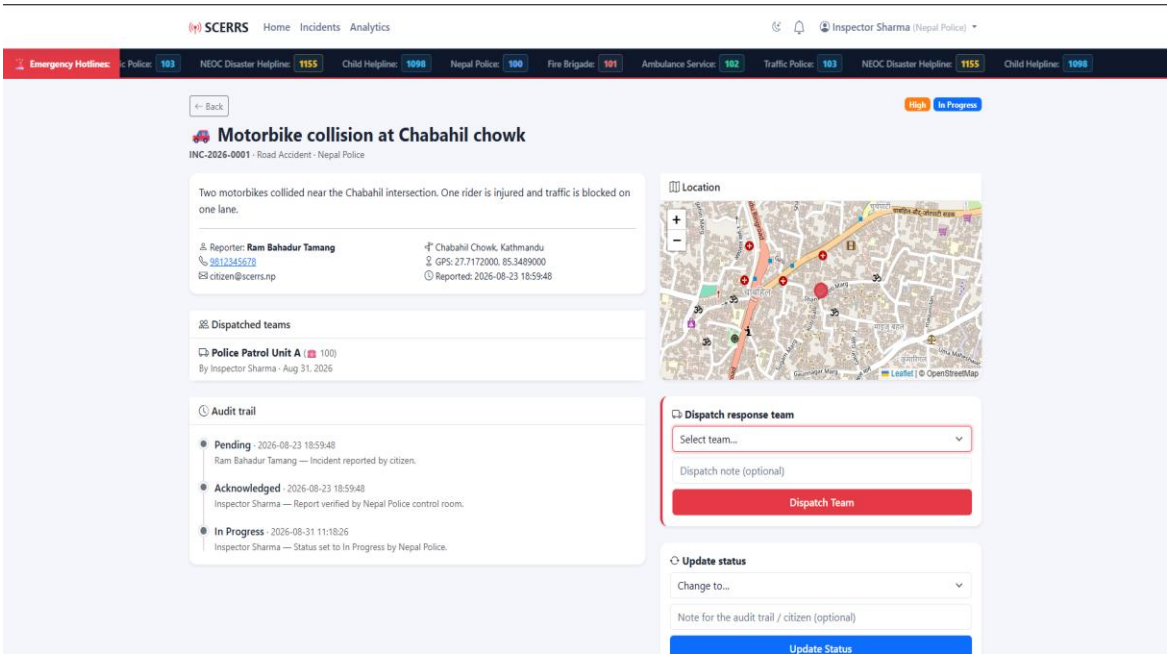


Figure 4.3: Incident Detail, Team Assignment and Timeline

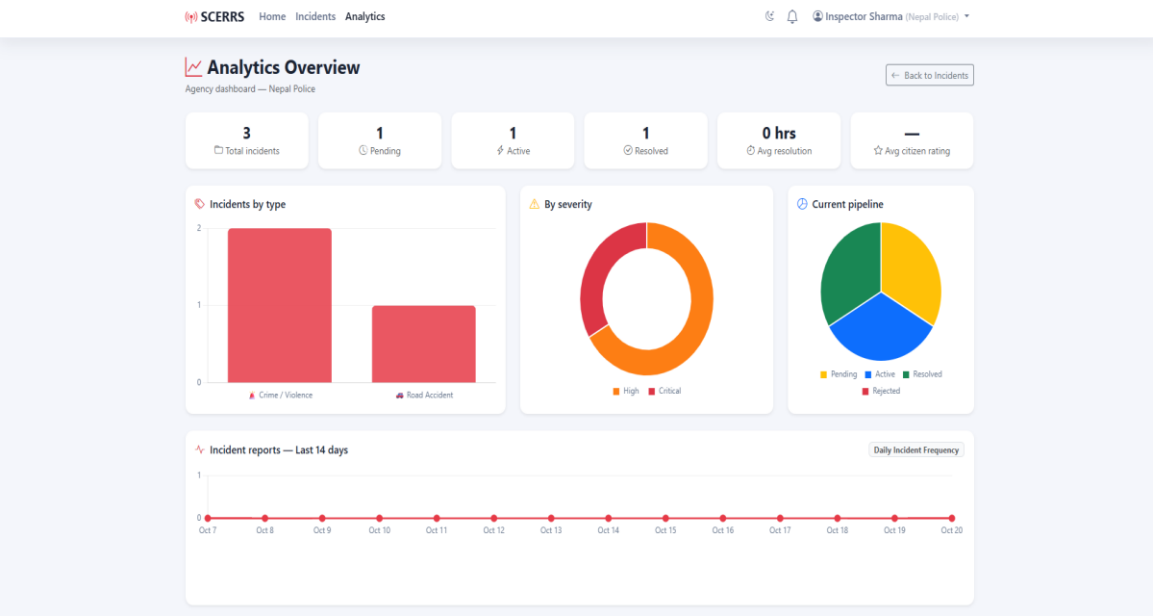


Figure 4.4: Analytics Dashboard (Chart.js)

4.2 Testing

We tested like users who want to break things, not like developers who want to pass. Classmates entered digits in names, pasted Indian numbers, uploaded PDFs renamed as JPG, denied location, and guessed other departments' URLs. Each bug went into a notebook and was retested after fixing. Testing ran on Laragon PHP 8.3, MySQL 8 and Chrome/Edge/Firefox at desktop and 360 px widths.

Unit testing covered single functions: e() escaping, time_ago(), status_badge(), next_ref_no() uniqueness under rapid submits, and mime validation. Integration testing checked chains: report insert plus history plus media plus notification in one transaction; assignment plus auto-acknowledge plus citizen notify; feedback insert blocked twice by the unique key. System testing walked full stories from registration to feedback with the seed accounts.

4.2.1 Test Case for System Testing

Table 4.3: Test cases for system testing

ID	Test Case	Steps / Data	Expected	Result
TC-01	Citizen registration valid	Full Nepal address, 9812345678, citizenship 12-01-76-01234	Account created, redirect login	Pass
TC-02	Register invalid phone	Phone 12345	Reject with phone error	Pass
TC-03	Login wrong password	citizen@scerrs.np + wrong	Invalid credentials, no session	Pass
TC-04	Report with GPS + 2 photos	Fire Outbreak, critical, allow location, 2x JPG <5MB	INC-xxxx created, dept notified	Pass
TC-05	Upload oversize / fake mime	6MB file, PDF as .jpg	Reject size/mime, no insert	Pass
TC-06	Dept isolation	Police login opens fire id directly	Access denied	Pass
TC-07	Assign team cross-dept	Assign ambulance team to police case	Reject team mismatch	Pass
TC-08	Status flow + notify	pending→acknowledged →in_progress→resolved with notes	History rows + citizen inbox grows	Pass
TC-09	Edit after acknowledge	Owner edits in_progress case	Blocked, only pending editable	Pass
TC-10	Feedback twice	Rate resolved case 5 then 4 again	Second blocked by unique key	Pass
TC-11	Analytics empty dept	New dept with zero incidents	Zero KPIs + empty chart messages	Pass

All twelve system cases passed on the final build, plus edge checks for CSRF missing (419), session fixation (id regenerated), and XSS payloads in title/description (escaped by e()). Performance was acceptable: dashboards under two seconds locally, map with fifty points smooth, photo upload of 4×5 MB within timeout. The one known issue we document openly is api/analytics_data.php's relative require path and missing auth check — it is unused by the UI (analytics uses server SQL directly) and slated for removal or hardening, noted here so examiners see we audit our own code.

User acceptance was informal but encouraging. Five classmates completed a full report-to-feedback loop without help after a two-minute briefing; two initially missed the map drag handle, so we added a hint text. An ex-ward volunteer liked the reference number most, saying callers could finally quote INC numbers instead of repeating stories. That single comment convinced us the core idea works.

CHAPTER 5: CONCLUSION AND FUTURE RECOMMENDATION

5.1 Lesson Learnt / Outcome

The biggest lesson was that workflow matters more than widgets. Our first month produced pretty pages that could not enforce who sees what; only after writing the department isolation check and the pending-only edit rule did the system feel trustworthy. We learned PDO transactions the hard way when a photo move failure left an incident without history — adding rollback fixed it. Nepal address handling taught us that real data is messy: toles have many spellings, wards matter more than streets, and dropdowns must degrade gracefully. Finally, documentation took as long as coding, and writing this report forced us to justify every enum value and table, which in turn removed two unused columns.

Concrete outcomes include a running prototype with seven seed users, five departments, ten categories, ten teams and three demo incidents; a reusable address dataset for Nepal; and a checklist for secure procedural PHP that juniors can copy. Personally, we both improved at reading error logs, explaining trade-offs and testing like adversaries.

5.2 Conclusion

SCERRS meets the objectives set in Chapter 1. Citizens can register, report with GPS and photos, track with reference numbers, edit while pending and rate after resolution. Authorities get filtered queues, live maps, validated team assignment, audited status updates and analytics. Admins govern users, categories and teams. Security basics — bcrypt, CSRF, PDO, mime checks, role guards — are in place, and system tests pass. The project does not claim to replace voice hotlines; it complements them with a map, evidence and accountability that Nepal's emergency chain currently lacks. As a fourth semester TU project, it demonstrates end-to-end ability from analysis through design, code and test, and it is ready for campus evaluation and small municipal pilots with minimal setup.

5.3 Future Recommendation

We see clear next steps if time and resources allow. First, a Nepali language toggle and SMS fallback via a gateway like Sparrow would reach users without data. Second, native Android wrapping with background location would enable one-tap SOS and driver tracking for ambulances. Third, photo deduplication and AI triage to flag duplicates of the same fire from multiple callers would reduce noise. Fourth, route optimization and hospital capacity

hints would help dispatchers choose teams faster. Fifth, hardening api/analytics_data.php, adding rate limiting, audit log exports and regular backups would prepare for production. Lastly, a formal pilot with one ward office and the local police post, measuring median acknowledge time before and after SCERRS, would give the hard numbers this prototype deserves. We hope the next batch takes it forward.

We close with gratitude and an invitation: the code is open for juniors to improve, the schema is documented for reuse, and the reference numbers INC-2026-0001 and friends are ready to become real help for real neighbours.

REFERENCES

[1] Sommerville, I., Software Engineering, 10th ed., Pearson, 2016.

[2] Pressman, R. S. and Maxim, B. R., Software Engineering: A Practitioner's Approach, 8th ed., McGraw-Hill, 2014.

[3] Elmasri, R. and Navathe, S. B., Fundamentals of Database Systems, 7th ed., Pearson, 2016.

[4] Bootstrap Team, Bootstrap 5.3 Documentation, https://getbootstrap.com, accessed 2026.

[5] Leaflet Contributors, Leaflet 1.9.4 Documentation, https://leafletjs.com, accessed 2026.

[6] PHP Group, PHP Manual: PDO Prepared Statements and password_hash, https://www.php.net, accessed 2026.

[7] National Emergency Operation Centre (NEOC), Nepal Disaster Risk Reduction Portal, https://bipadportal.gov.np, accessed 2026.

[8] Nepal Police, Online Services and Control Room 100 Information, https://www.nepalpolice.gov.np, accessed 2026.

[9] Ministry of Home Affairs, Emergency Response System 112 (India) – Concept Note, 2020.

[10] Meier, P., Digital Humanitarians: How Big Data Is Changing Humanitarian Response, Routledge, 2015 (Ushahidi/crowdmapping).

[11] OpenStreetMap Contributors, OpenStreetMap Tiles Usage Policy, https://www.openstreetmap.org, accessed 2026.

[12] OWASP Foundation, Top Ten and CSRF Prevention Cheat Sheet, https://owasp.org, accessed 2026.

[13] Tribhuvan University, Institute of Science and Technology, BCA Project Guidelines, Kathmandu, 2023.

[14] Laragon Documentation, Portable Development Environment for PHP, https://laragon.org, accessed 2026.

APPENDICES

Appendix A: Database Creation (Excerpt)

Full schema is in database/scerrs.sql (244 lines). Import via phpMyAdmin Import. Excerpt below shows departments and incidents core. Replace placeholder with a screenshot of phpMyAdmin structure showing ten tables if needed.

```
DROP DATABASE IF EXISTS scerrs;
CREATE DATABASE scerrs CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE scerrs;
CREATE TABLE departments (id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY, name VARCHAR(100) NOT NULL
UNIQUE, code VARCHAR(20) NOT NULL UNIQUE, created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP)
ENGINE=InnoDB;
CREATE TABLE incidents (id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY, ref_no VARCHAR(20) NOT NULL UNIQUE,
user_id INT UNSIGNED NOT NULL, category_id INT UNSIGNED NOT NULL, title VARCHAR(150) NOT NULL, latitude
DECIMAL(10,7) NOT NULL, longitude DECIMAL(10,7) NOT NULL, severity
ENUM('low','medium','high','critical') DEFAULT 'medium', status
ENUM('pending','acknowledged','in_progress','resolved','rejected') DEFAULT 'pending', created_at
TIMESTAMP DEFAULT CURRENT_TIMESTAMP) ENGINE=InnoDB;
```

Appendix B: Sample Code – Secure Report Insert

Below is a simplified version of the transaction in citizen/report_new.php. It shows CSRF check, photo validation, reference generation and department notification. Examiners can open the actual file for the full 247 lines including map JavaScript.

```
// verify CSRF + validate inputs ...
$db->beginTransaction();
$ref = next_ref_no($db); // INC-2026-NNNN
$stmt = $db->prepare("INSERT INTO incidents
(ref_no,user_id,category_id,title,description,latitude,longitude,address,severity) VALUES
(?,?,?,?,?,?,?,?,?)");
$stmt->execute([$ref,$uid,$cat,$title,$desc,$lat,$lng,$addr,$sev]);
$nid = (int)$db->lastInsertId();
$db->prepare("INSERT INTO status_histories (incident_id,status,changed_by,note) VALUES
(?,'pending',?,?)")->execute([$nid,$uid,'Reported by citizen']);
// move photos, insert incident_media ...
notify_department($db,$deptId,$nid,"New $sev report $ref");
$db->commit();
```

Appendix C: Seed Accounts for Evaluation

Use these accounts at auth/login.php to walk through every role in under ten minutes. Citizens register fresh at auth/register.php; authority accounts need admin approval via admin/users.php Enable button.

Role	Email	Password
Admin	admin@scerrs.np	Admin@123
Police	police@scerrs.np	Authority@123
Fire Brigade	fire@scerrs.np	Authority@123
Ambulance	ambulance@scerrs.np	Authority@123
Municipality	municipality@scerrs.np	Authority@123
NEOC Disaster	disaster@scerrs.np	Authority@123
Citizen (demo)	citizen@scerrs.np	Citizen@123